MP5: Kernel-Level Thread Scheduling

Mauricio Aguilar Alonso
UIN: 434002903
CSCE 410/611: Operating Systems

Assigned Tasks

Main assignment: Completed. FIFO scheduling and terminating thread functions.
Bonus option 1: Interrupt management. Not completed.
Bonus option 2: Round-robin scheduling. Not completed.
Bonus option 3: Processes with separate address spaces. Not completed.

System Design

A singly linked list fits the two operations needed for first-in, first-out (FIFO) scheduling: threads join at the tail and leave from the head. Keeping both pointers lets the scheduler update either end directly. The list can grow with the number of ready threads without choosing an array size in advance. The tradeoff is one node allocation each time a thread joins the queue. Since the supplied memory pool does not reuse released nodes, repeated handoffs consume memory over time.

A thread still needs its stack to give up the CPU, even after its thread function returns. The scheduler therefore saves a pointer to the finished thread so another thread can clean it up after the context switch. An idle thread provides a separate stack even when the last runnable thread finishes.

Code Description

The implementation changes are in `scheduler.H`, `scheduler.C`, `thread.H`, and `thread.C`. The provided macros in `kernel.C` select the test configuration.

`scheduler.H` defines the queue node inside the scheduler so the links can be maintained without changing the thread control block. The header also declares the idle and finished thread pointers and the helpers for shutdown and cleanup. The public methods listed in the handout keep their original signatures.

```
class Scheduler {

   /* Ready Queue members*/
   struct Node {
      Thread * thread;
      Node * next;
   };
   Node * ready_head;
   Node * ready_tail;
   Thread * idle;
   Thread * finished;
   void collect_finished();

public:
   static void terminate_current();
   /* Entry from thread_shutdown when a thread function returns. */
```

Ready queue members and helper declarations in `scheduler.H`.

`thread.H` declares a destructor because each thread owns a separately allocated stack. This lets the scheduler delete a finished thread while keeping its stack pointer private.

scheduler.C: Scheduler() The supplied kernel uses one scheduler. Recording it at construction gives every returning thread a way to reach it without passing a scheduler argument to each thread function.

```
Scheduler::Scheduler() : ready_head(0), ready_tail(0), idle(0), finished(0) {
  active_scheduler = this;
  Console::puts("Constructed Scheduler.\n");
}
```

Initializing the scheduler.

scheduler.C: resume() A ready thread should have only one place in line. A duplicate entry could give it an extra turn or leave a pointer to it after it exits. Checking for duplicates costs a scan of the queue, but the tail pointer makes the insertion itself constant time.

```
void Scheduler::resume(Thread * _thread) {
  /* A thread may only appear in the ready queue once. */
  assert(_thread);
  for (Node * queued = ready_head; queued; queued = queued->next) {
    assert(queued->thread != _thread);
  }
  Node * node = new Node{_thread, 0};
  if (ready_tail) {
    ready_tail->next = node;
  } else {
    ready_head = node;
  }
  ready_tail = node;
}
```

Appending a thread to the ready queue.

scheduler.C: add() A newly created thread waits behind the threads that are already ready. Calling `resume()` gives new threads the same ordering and duplicate check as threads waiting for another turn.

```
void Scheduler::add(Thread * _thread) {
  resume(_thread);
}
```

Adding a newly created thread.

scheduler.C: yield() Giving up the CPU does not always mean a thread wants another turn. The kernel already puts a continuing thread back in line before it yields. Keeping that decision outside this method lets a finished thread use the same handoff without being scheduled again.



```
void Scheduler::yield() {
  collect_finished();
  if (!ready_head) {
    /* Keep the idle thread outside the FIFO queue. */
    if (!idle) idle = new Thread(run_idle, new char[1024], 1024);
    if (Thread::CurrentThread() != idle) Thread::dispatch_to(idle);
    return;
  }

  Node * node = ready_head;
  Thread * next = node->thread;
  ready_head = node->next;
  if (!ready_head) ready_tail = 0;
  delete node;

  if (next != Thread::CurrentThread()) {
    Thread::dispatch_to(next);
  }
}
```

Selecting the next thread to run.

scheduler.C: run_idle() *Added for this implementation.* The idle thread stays outside the ready queue so it never takes a turn ahead of waiting work. Its loop yields, giving the scheduler a chance to clean up the final thread and check whether work is ready.

```
static void run_idle() {
  for (;;) active_scheduler->yield();
}
```

The idle thread function.

thread.C: thread_shutdown() The initial stack already contains this function's address. Using that return path gives every thread the same shutdown behavior, so an ordinary return is enough to finish a thread.

```
static void thread_shutdown() {
    /* setup_context placed this return address above the thread function. */
    Scheduler::terminate_current();
}
```

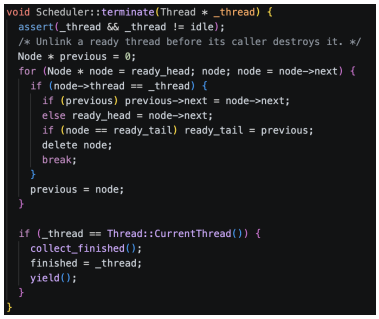
Entering shutdown when a thread function returns.

scheduler.C: terminate_current() *Added for this implementation.* Shutdown receives no thread argument. This helper asks the dispatcher which thread is running and passes it to the active scheduler. The scheduler therefore has no second current-thread pointer to keep in sync.

```
void Scheduler::terminate_current() {
  assert(active_scheduler);
  active_scheduler->terminate(Thread::CurrentThread());
}
```

Passing the running thread to the scheduler.

scheduler.C: terminate() A queued thread can be removed immediately because it is waiting for the CPU. A thread terminating itself must leave its stack intact for the handoff. Each exiting thread collects any earlier exit before recording its own, so two consecutive exits cannot overwrite the pointer to a thread awaiting cleanup.



```
void Scheduler::terminate(Thread * _thread) {
  assert(_thread && _thread != idle);
  /* Unlink a ready thread before its caller destroys it. */
  Node * previous = 0;
  for (Node * node = ready_head; node; node = node->next) {
    if (node->thread == _thread) {
      if (previous) previous->next = node->next;
      else ready_head = node->next;
      if (node == ready_tail) ready_tail = previous;
      delete node;
      break;
    }
    previous = node;
  }

  if (_thread == Thread::CurrentThread()) {
    collect_finished();
    finished = _thread;
    yield();
  }
}
```

Removing a thread from scheduling.

scheduler.C: collect_finished() *Added for this implementation.* The first cleanup check can occur in the exiting thread itself, since it calls `yield()` to give up the CPU. Comparing `finished` with the running thread ensures deletion happens only after execution has moved to another stack.

```
void Scheduler::collect_finished() {
  /* Only allow a thread to release its predecessor only after switching stacks. */
  if (finished && finished != Thread::CurrentThread()) {
    delete finished;
    finished = 0;
  }
}
```

Checking that execution has switched to another stack.

thread.C: `~Thread()` *Added for this implementation.* The kernel creates each stack with `new[]`. Deleting the thread object alone would leave that allocation behind, so its destructor releases the stack with the matching `delete[]`.



Releasing the finished thread's stack.

Testing

Testing used the `_USES_SCHEDULER_` and `_TERMINATING_FUNCTIONS_` macros in `kernel.C`.